Industrial Digital Twin
Transformer Health Monitoring System

CO326 – Course Project Documentation
Group 24

April 2026 | University of Peradeniya, Dept. of Computer Engineering

Group Members

Student ID	Name	Role
E/20/009	Ahemad I.I	Edge AI & Simulation
E/20/199	Ketharagan P	Node-RED & RUL
E/20/339	Rismy A.M	Infrastructure & Security
E/20/371	Rishopika S	Documentation & Dashboards

Table of Contents

- 1. System Architecture Diagram
- 2. Electrical Wiring Diagram
- 3. Simplified P&ID
- 4. MQTT Topic Hierarchy
- 5. Node-RED Flow Export
- 6. ML Model Description
- 7. Cybersecurity Design Summary

1. System Architecture Diagram

The system follows a four-layer Industrial IoT reference architecture as mandated by the CO326 project guidelines. Each layer has clearly defined responsibilities and communicates with adjacent layers through standardised protocols.

1.1 Architecture Overview

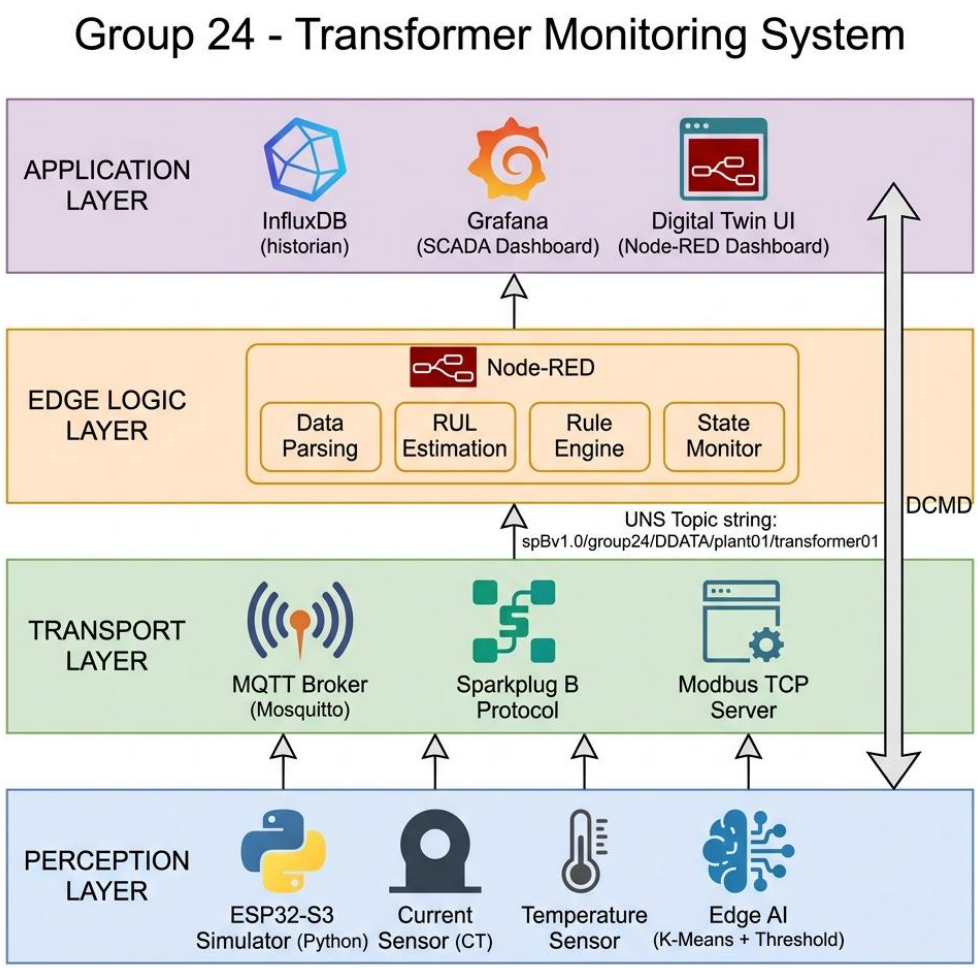

Figure 1 – System Architecture Diagram

1.2 Layer Descriptions

Layer	Description
Layer 1 – Perception	ESP32-S3 edge node (simulated via Python). Reads load current (CT sensor) and winding temperature (NTC/thermocouple). Runs on-device Edge AI (K-Means + statistical thresholding) and exposes Modbus TCP registers.
Layer 2 – Transport	Eclipse Mosquitto MQTT broker (v2.0). Uses Sparkplug B topic structure (DBIRTH, DDATA, DDEATH, DCMD) over the Unified Namespace spBv1.0/group24/…. QoS 1, authentication enforced, ACL restricted.
Layer 3 – Edge Logic	Node-RED flow engine. Parses Sparkplug B payloads, executes RUL estimation (linear regression on temperature history), monitors state consistency, and routes data to storage.
Layer 4 – Application	InfluxDB 1.8 time-series historian + Grafana SCADA dashboards + Node-RED Dashboard Digital Twin UI. Supports bidirectional control via DCMD topics.

1.3 Technology Stack

Component	Technology	Version / Notes
Edge Simulator	Python 3.9+	NumPy, scikit-learn, paho-mqtt
MQTT Broker	Eclipse Mosquitto	2.0 – auth + ACL enabled
Flow Engine	Node-RED	Latest – node-red-dashboard, influxdb
Time-Series DB	InfluxDB	1.8 – measurement: transformer_data
Visualisation	Grafana	Latest – auto-provisioned datasource
Containerisation	Docker / Docker Compose	docker-compose v3.8
Modbus	pymodbus	TCP server on port 502

2. Electrical Wiring Diagram

The wiring diagram illustrates how the physical sensors connect to the ESP32-S3 microcontroller, which acts as the edge node for the distribution transformer monitoring system.

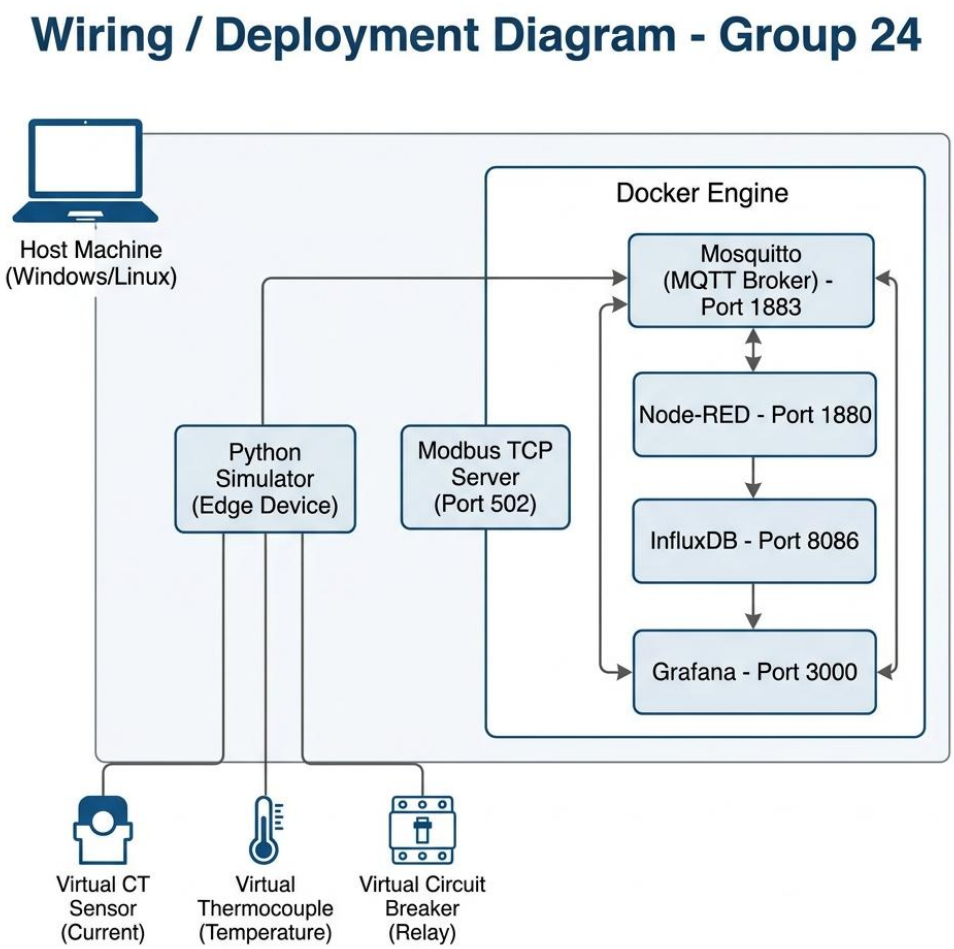

Figure 2 – Electrical Wiring Diagram

2.1 Sensor Connections

Sensor	Type	ESP32-S3 Pin	Protocol	Measurement
Current Transformer (CT)	Analog CT sensor	GPIO 34 (ADC1_CH6)	Analog/ADC	Load current RMS (A)
Temperature Sensor	NTC Thermistor / Thermocouple	GPIO 32 (ADC1_CH4)	Analog / I2C	Winding temp (°C)
Circuit Breaker Actuator	Relay module	GPIO 26 (Digital Out)	Digital GPIO	Trip / Reset command
Status LED	LED indicator	GPIO 2	Digital Out	Visual anomaly alert

2.2 Communication Interfaces

Interface	Protocol	Purpose
Wi-Fi (802.11 b/g/n)	MQTT over TCP/IP	Upload telemetry to broker; receive DCMD commands
Modbus TCP	Modbus over Ethernet	Expose register map to SCADA / legacy systems
ADC (12-bit)	Analog	Read raw sensor voltages from CT and thermistor
GPIO (digital)	Digital I/O	Drive relay actuator; read discrete inputs

3. Simplified P&ID (Piping & Instrumentation Diagram)

The P&ID shows the measurement points, instrumentation tags, and control loops for the distribution transformer monitoring system.

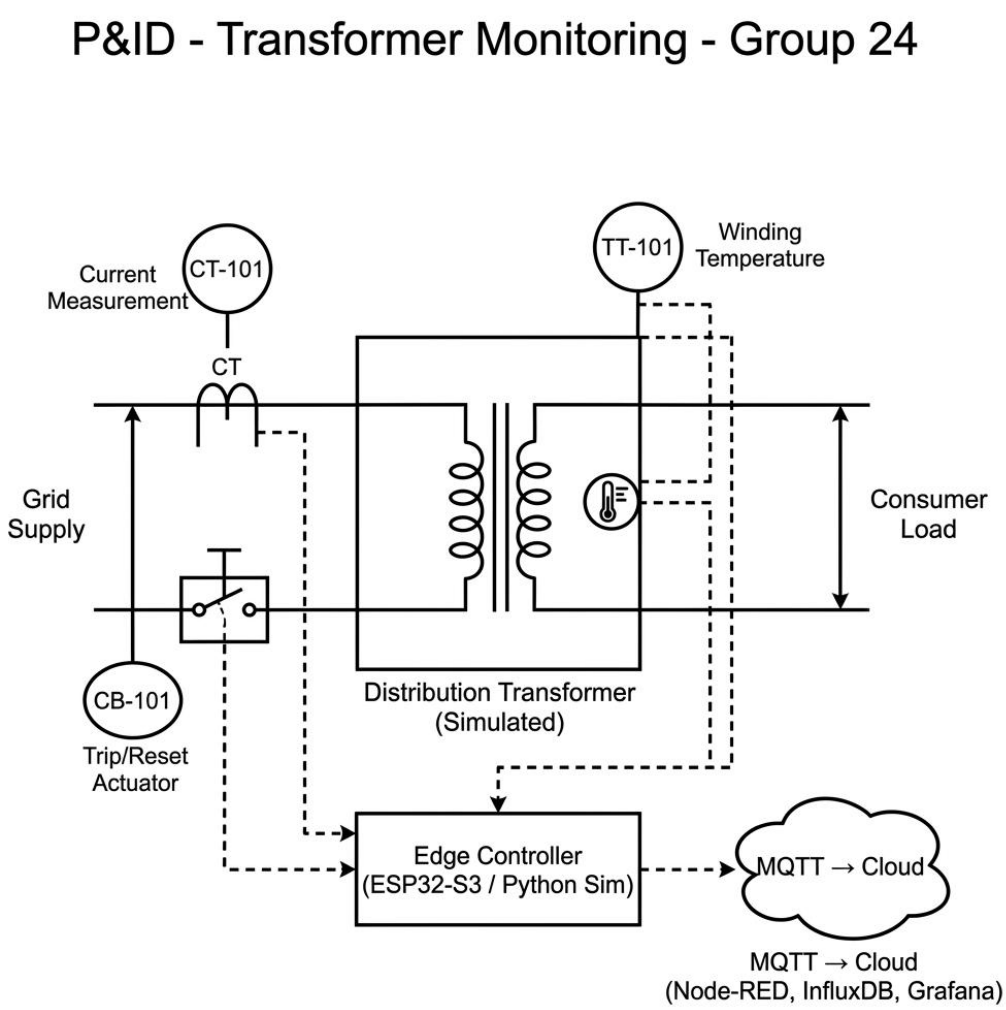

Figure 3 – Simplified P&ID Diagram

3.1 Instrumentation Tag List

Tag	Description	Range	Alarm High	Action
IT-101	Load Current Transmitter	0 – 30 A	15 A	Trigger anomaly; consider trip
TT-101	Winding Temperature Transmitter	0 – 120 °C	85 °C	Trigger anomaly; consider trip
AS-101	Anomaly Score (Edge AI output)	0.0 – 1.0	1.0	Alert dashboard; increment RUL counter
ZSC-101	Circuit Breaker / Actuator Status	0 = Normal, 1 = Tripped	–	Reported via MQTT ActuatorState metric

RUL-101	Remaining Useful Life Estimator	0 – ∞ hours	0 h (CRITICAL)	Dashboard warning; Grafana alert

3.2 Control Loops

The system implements the following control loops:

- Anomaly Detection Loop: Edge AI continuously evaluates [IT-101, TT-101] → publishes AnomalyScore → Node-RED state monitor evaluates → dashboard alerts operator.
- Actuator Control Loop (bidirectional): Operator sends ActuatorCommand via DCMD topic → simulator receives → trips/resets circuit breaker relay → reports ActuatorState back via DDATA.
- What-If Simulation Loop: Operator adjusts WhatIfLoad slider → DCMD overrides simulated current → system re-evaluates anomaly and RUL under hypothetical load.
- RUL Estimation Loop: Node-RED maintains 60-sample temperature history → linear regression → projects time to 85°C threshold → displays hours remaining.

4. MQTT Topic Hierarchy

The system implements the Sparkplug B (spBv1.0) Unified Namespace convention as the primary topic structure, supplemented by a simple status topic.

4.1 Topic Tree

```
spBv1.0/
└── group24/                        # Group ID
    ├── DBIRTH/plant01/transformer01  # Birth certificate (on connect)
    ├── DDATA/ plant01/transformer01  # Telemetry (every 2 s)
    ├── DDEATH/plant01/transformer01  # Death cert / LWT (on disconnect)
    └── DCMD/  plant01/transformer01  # Commands (cloud → edge)

status/group24/transformer01          # online | offline (retained)
modbus/group24/transformer01/registers # Modbus register mirror (Node-RED)
```

4.2 Topic Reference Table

Topic	Direction	QoS	Retain	Purpose
spBv1.0/group24/DBIRTH/plant01/transformer01	Edge → Cloud	1	Yes	Birth certificate – lists all metrics
spBv1.0/group24/DDATA/plant01/transformer01	Edge → Cloud	1	No	Sensor telemetry (2 s interval)
spBv1.0/group24/DDEATH/plant01/transformer01	Edge → Cloud	1	Yes	Death certificate / LWT
spBv1.0/group24/DCMD/plant01/transformer01	Cloud → Edge	1	No	Actuator & What-If commands
status/group24/transformer01	Edge → Cloud	1	Yes	Simple online/offline flag

4.3 DDATA Payload Structure

```
{
  "timestamp": "2026-04-30T10:15:30.123456+00:00",
  "seq": 42,
  "metrics": [
    {"name": "Current",       "value": 10.42, "type": "Float",  "unit": "A"},
    {"name": "Temperature",   "value": 33.1,  "type": "Float",  "unit": "degC"},
    {"name": "AnomalyScore",  "value": 0.0,   "type": "Float"},
    {"name": "AnomalyMethod","value": "kmeans+threshold", "type": "String"},
    {"name": "ActuatorState","value": 0,     "type": "Int32"},
    {"name": "UptimeSeconds","value": 120,   "type": "Int64"}
  ]
}
```

5. Node-RED Flow Export

The Node-RED flow (docs/flows.json) implements Layers 3 and 4 of the architecture. It is exported as JSON and can be imported directly via Node-RED Menu → Import.

5.1 Flow Overview

The flow contains one tab labelled 'Digital Twin - Group 24' with the following functional groups:

Node / Group	Description
MQTT Subscribers	Four MQTT-in nodes subscribing to DDATA, DBIRTH, DDEATH, and status topics.
Parse Sparkplug B Metrics	Function node that flattens the metrics array into a key-value object for downstream processing.
RUL Estimator	Function node maintaining a 60-sample temperature history and computing linear regression to estimate hours-to-failure.
State Consistency Monitor	Function node detecting and alerting when anomaly is flagged but actuator is not tripped.
Device Offline Alert	Function node triggered on DDEATH to warn that the device has disconnected.
InfluxDB Writers	Two influxdb-out nodes writing transformer_data and rul_estimation measurements.
Dashboard UI	Gauges for Current and Temperature, text indicators for Anomaly and Actuator state, RUL text display.
Bidirectional Control	Trip button, Reset button, and What-If slider – all publishing to the DCMD topic.

5.2 Key Function Nodes

RUL Estimator (Linear Regression)

Maintains history of up to 60 temperature readings (2-minute window). Applies least-squares linear regression. If slope > 0.01 °C/sample, projects time until 85 °C threshold is reached.

```
slope = (n*ΣXY - ΣX*ΣY) / (n*ΣX² - (ΣX)²)
slopePerHour = slope × (3600 / 2)   // 2-second sample interval
RUL_hours = (85 - currentTemp) / slopePerHour
```

State Consistency Monitor

Rule: if AnomalyScore > 0 AND ActuatorState == 0 → raise WARNING via node.warn() and output alert payload to debug sidebar.

5.3 Import Instructions

- 1. Open Node-RED: http://localhost:1880
- 2. Click the hamburger menu (☰) in the top-right corner.
- 3. Select Import → Clipboard.
- 4. Paste the contents of docs/flows.json.
- 5. Click Import, then Deploy.
- 6. Install palettes via Manage Palette → Install: node-red-dashboard, node-red-contrib-influxdb.

6. ML Model Description

The Edge AI module runs entirely on the edge node (simulated ESP32-S3) using a hybrid anomaly detection strategy combining statistical thresholding and unsupervised K-Means clustering.

6.1 Strategy Overview

Method	Always Active?	Training Required?	Detects
Statistical Thresholding	Yes	No	Hard limit violations (spike anomalies)
K-Means Clustering	After training	Yes (100 normal samples)	Subtle drift / pattern deviations
Combined (max)	Yes	Partial	Either method flags → anomaly score = 1.0

6.2 Statistical Thresholding

Parameter	Threshold	Standard / Rationale
Temperature	> 85.0 °C	IEC 60076 winding hot-spot limit
Current	> 15.0 A	Rated current × 1.5 overload factor

6.3 K-Means Clustering

Algorithm: MiniBatchKMeans | Clusters (k): 3 | Training samples: 100 normal-operation readings | Features: [Current (A), Temperature (°C)]

Anomaly threshold: mean(d) + 2.0 × std(d), where d = Euclidean distance to nearest cluster centroid.

Model Lifecycle

```
Startup          →  Threshold-only mode
100 normal samples collected  →  K-Means fitted
Inference: distance to nearest centroid → compare to distance_threshold
combined_score = max(threshold_score, kmeans_score)
```

6.4 ESP32-S3 Production Deployment Notes

- Centroids (3 × 2 float32 array) trained offline on historical data.
- Exported as C header file and hard-coded into firmware.
- Inference = 6 multiplications + 3 additions + 3 comparisons (< 1 µs at 240 MHz).
- Memory footprint: ~24 bytes (3 centroids × 2 features × 4 bytes/float).
- No ML library dependency on device – pure arithmetic in C.

6.5 MQTT Anomaly Payload

```
{"name": "AnomalyScore",  "value": 1.0, "type": "Float"}
{"name": "AnomalyMethod", "value": "kmeans+threshold", "type": "String"}
```

7. Cybersecurity Design Summary

The cybersecurity design addresses authentication, authorisation, credential protection, device lifecycle management, and data reliability. All measures align with Sections 10 and 11 of the CO326 project guidelines.

7.1 Security Architecture

```
┌──────────────────── Mosquitto MQTT Broker ────────────────────┐
│  Authentication (passwd/PBKDF2-SHA512) │ ACL Engine │ LWT      │
└───────────────────────────────────────────────────────────────┘

        ▲                    ▲                    ▲
   Edge Simulator      Node-RED Flows       Grafana Dashboard
```

7.2 Authentication

Aspect	Implementation
Method	Username / password per MQTT client
Password storage	Mosquitto passwd file – PBKDF2-SHA512 hashed
Anonymous access	Disabled (allow_anonymous false)
Credential storage	.env file, excluded from Git via .gitignore
Runtime injection	Docker Compose ${VARIABLE} environment substitution

7.3 MQTT Users

Username	Role	Permissions
group24	Edge device	Publish spBv1.0/group24/#, status/group24/#; Subscribe DCMD
nodered	Flow engine	Full read/write on group24 topics and modbus/group24/#

7.4 ACL – Principle of Least Privilege

```
user group24
topic readwrite spBv1.0/group24/#
topic readwrite status/group24/#
topic read      spBv1.0/group24/DCMD/#

user nodered
topic readwrite spBv1.0/group24/#
topic readwrite status/group24/#
topic readwrite modbus/group24/#
```

7.5 Reliability Features

Feature	Implementation
Last Will & Testament (LWT)	DDEATH published to spBv1.0/group24/DDEATH/… on unexpected disconnect (QoS 1, retained)
Local Data Buffering	buffer_offline.json stores messages when broker unreachable; replayed on reconnect
Exponential Backoff Reconnection	Initial 1 s delay, ×2 per attempt, max 60 s
Timestamped Data	UTC ISO-8601 timestamp on every MQTT message; Sparkplug B sequence numbers
State Consistency Monitoring	Node-RED detects actuator/anomaly state drift and raises alerts

7.6 Threat Model

Threat	Mitigation
Unauthorised MQTT access	Username/password authentication, anonymous disabled
Topic hijacking / eavesdropping	ACL-based topic access control
Credential leakage	.env + .gitignore; no secrets in source code
Device failure / silent disconnect	LWT + DDEATH Sparkplug B message
Network outage data loss	Local file buffer + replay on reconnection
Stale or duplicate data	Sequence numbers (seq) + UTC timestamps
State inconsistency	Node-RED state consistency monitor function node

7.7 Production Recommendations

- TLS encryption on MQTT (port 8883) with X.509 certificates.
- Mutual TLS (mTLS) for edge device certificate-based authentication.
- Role-Based Access Control (RBAC) with a central identity provider.
- Audit logging of all MQTT connections and command executions.
- Network segmentation between OT and IT zones (Purdue Model / IEC 62443).
- VPN or private network for MQTT broker exposure.